ADVANCED ENGINEERING SPECIFICATION

Multiplayer Ludo Module

Cospira Rooms Platform Integration

Real-time · Host-authoritative · Production-grade

ROLE	BASE REPO	VERSION
Senior Full-Stack Engineer	`github/tokyRT/Ludo-mg`	**v2.0 — includes W1/W2/W3 patches**

1 Repository analysis & architectural decoupling

Clone tokyRT/Ludo-mg and perform a deep architectural audit. Identify every concern and map it to one of four isolated layers. No layer may import from a layer above it. The State layer has zero UI dependencies.

Rendering layer	Rules layer	Interaction layer	State layer
Canvas/DOM draw calls, animation, sprite sheets	Legal move validation, win conditions, safe squares	Input events, dice UI, token selection UX	Turn order, token positions, game lifecycle FSM

2 Modular engine — file structure & contracts

Scaffold the following module under `/cospira/modules/games/ludo/`. Each file must export a typed interface so future games can swap implementations:

`LudoEngine.js`	Game lifecycle FSM, turn orchestration, event bus publisher
`DiceEngine.js`	Cryptographically seeded RNG, roll result serialization
`TokenController.js`	Path resolution, capture logic, home-stretch entry rules
`GameStateManager.js`	Immutable state store, optimistic update support, snapshot/restore
`RulesEngine.js`	Pure validator — takes (state, action) → {valid, reason}
`LudoBoard.jsx`	React renderer — subscribes to state, zero business logic

3 High-fidelity UI/UX — board & room integration

Entry flow

Room → Activities → Games → Ludo → **Start game**

Board polish requirements

- SVG-based board — crisp at any resolution, no bitmap assets
- Token movement: cubic-bezier eased path animation, 300ms per cell, queued not concurrent
- Dice: CSS 3D flip animation with tactile micro-bounce on land, seeded value revealed on bounce end

- Capture flash: brief pulse on captured token before it returns to yard
- Safe squares visually distinct (subtle star glyph, no color clash with player tokens)
- Responsive — scales from 320px mobile to 1440px desktop via SVG viewBox
- Dark mode: all board colors via CSS custom properties, no hardcoded hex on the board surface
- Active player panel glows with a soft 2px accent ring in their color, no distracting animations

Room layout — game active state

Voice stays active. Chat collapses to sidebar. Board slides in with a 250ms ease-out scale from 0.96.

voice panel	**game board (center)**	chat panel

4 Real-time sync — latency budget & event design

Extend Cospira's existing WebSocket channel. Do not introduce a second connection. All game messages flow through a `GAME_EVENT` envelope:

```
{
  "type": "GAME_EVENT",
  "game": "ludo",
  "roomId": "room_abc123",
  "senderId": "user_45",
  "seq": 142,            // monotonic — drop if seq < last seen
  "action": "MOVE_TOKEN",
  "payload": {
    "tokenId": "red_2",
    "fromCell": 14,
    "toCell": 18
  },
  "ts": 1712000000000   // sender timestamp for RTT display
}
```

Latency targets
Dice broadcast: <80ms p95
Token animation start: <150ms p95
Turn switch: <200ms p95

Resilience
Sequence gaps: request full state resync
Disconnect mid-turn: 30s reconnect window
Host crash: deterministic host re-election

5 Host-authoritative model — cheat prevention

Client	— sends REQUEST_ROLL → host only
Host	— validates turn ownership, generates roll, broadcasts ROLL_RESULT
All clients	— receive roll, animate dice, await MOVE_TOKEN

RulesEngine runs on host only. Clients render. Client-originated state mutations are silently dropped. If host and client state diverge, client requests STATE_SYNC.

6 Game state schema (canonical)

```
{
  version: 3,
  roomId: "room_abc",
  hostId: "user_01",
  secondaryHostId: "user_02",       // W1: hot standby reference
  hostHeartbeatTs: 1712000000000,   // W1: updated every 1s by host
  electionInProgress: false,        // W1: prevents double-election races
  status: "TURN_ACTIVE",            // FSM state
  players: [
    { id: "user_01", color: "red",    connected: true,  rank: null },
    { id: "user_02", color: "blue",   connected: true,  rank: null },
    { id: "user_03", color: "green",  connected: false, rank: null },
    { id: "user_04", color: "yellow", connected: true,  rank: null }
  ],
  currentTurn: "user_02",
  turnDeadlineTs: 1712000030000,    // 30s turn timer
  diceValue: 4,
  diceRolled: true,
  tokens: {
    red:    [  0,  0,  5, 10 ],     // -1 = in yard, 57 = home
    blue:   [  3,  7,  0,  0 ],
    green:  [ -1, -1, -1, -1 ],
    yellow: [  2, -1, -1, -1 ]
  },
  moveHistory: [],                  // last 10 moves for replay
  spectators: ["user_05"],
  seq: 144,                         // W2: authoritative global seq counter
  lastSnapshotSeq: 120,             // W2: seq of last full STATE_SYNC sent
  // touchLayout: "tray"|"direct"   // W3: client-local only, never synced
}
```

7 UX details — the things that make it feel polished

Turn awareness

Idle player tokens desaturated when not their turn
30s countdown ring on active player avatar
Turn skip if timer expires — visible toast to all
"Your turn" audio cue + tab title badge

Feedback & micro-interactions

Selectable tokens pulse gently on valid-move state
Invalid move: brief shake + muted error sound
Capture: victim token slides back to yard with sound
Win: confetti burst + scoreboard overlay

Spectator mode

Spectators see the board in read-only state
Spectator count shown on game panel header
Spectators can react with emoji reactions

Accessibility

All interactive elements keyboard-navigable
Tokens have aria-label with position context
Color-blind mode: shape + pattern to differentiate players

8 Performance contract

60	<2KB	delta	rAF
FPS target	per event payload	sync only (no full state)	requestAnimationFrame gating

Token animation uses `requestAnimationFrame` exclusively — never `setTimeout`/`setInterval`. State updates are batched with React's concurrent mode. Board re-renders are gated by shallow-equal on tokens only.

9 Security — server-side validation checklist

- ✗ Client-originated dice values
- ✗ Moving another player's token
- ✗ Duplicate ROLL requests in same turn
- ✗ Out-of-turn action requests
- ✗ Token position desync manipulation
- ✗ Spectator sending game actions

All violations return a REJECTED_ACTION event visible only to the sender.
Log to server audit trail with playerId + timestamp.

10 Extensibility — game adapter contract

Every future game must implement this interface. Ludo is the reference implementation.

```
interface GameEngineAdapter {
  init(players: Player[], roomId: string): GameState
  handleAction(state: GameState, action: GameAction): GameState
  validate(state: GameState, action: GameAction): ValidationResult
  isTerminal(state: GameState): boolean
  getWinner(state: GameState): Player | null
}

interface BoardUI {
  render(state: GameState): JSX.Element
  onInteraction(callback: (action: GameAction) => void): void
}
```

Future games planned: Chess · UNO · Poker · Monopoly. Each implements the same adapter — plug and play.

PATCH DOCUMENT — V2 CRITICAL FIXES

Three production-killing gaps — fully specified

Host reliability · Packet ordering · Mobile touch UX

W1 Host reliability — deterministic election & hot standby

The original spec assumed the host stays alive. If the host disconnects, dice generation stops, the rules engine vanishes, and the game freezes for all remaining players. This section fully defines how that is prevented and recovered from.

Election rule (primary)

```
host = players.sortBy(p => p.joinTimestamp)[0]

If that player disconnects:
  host = players.filter(p => p.connected).sortBy(p => p.joinTimestamp)[0]

This is deterministic — all clients compute the same result
independently without needing a coordination message.
```

Hot standby (secondary host)

```
primaryHost   = players[0]   // lowest joinTimestamp
secondaryHost = players[1]   // next lowest

secondaryHost shadows the game state identically to primary.
On primary disconnect:
  1. secondaryHost detects absence via heartbeat timeout (3s)
  2. secondaryHost promotes itself — broadcasts HOST_ELECTED
  3. All clients update their hostId reference
  4. New secondary = players[2] (or null if 2-player game)

Heartbeat: primary → all clients, every 1s.
If a client sees no heartbeat for 3s, it triggers election.
```

State to transfer on election

Secondary already has	Clients without standby
Full game state mirror (tokens, scores, turn order, dice history). No transfer needed — election is instant.	New host broadcasts STATE_SYNC immediately after HOST_ELECTED. Clients apply it as ground truth and resume.

Reconnect window

```
If original primary reconnects within 30s:
```

```
  → rejoins as a regular player (not reclaiming host)
  → receives STATE_SYNC from current host
  → resumes from current turn

If original primary reconnects after 30s:
  → game has already continued or ended
  → player rejoins as spectator only
```

Every client must implement the same election function — no coordination race is possible. Rule: lowest join timestamp among currently connected players wins. All clients compute this identically.

W2 Packet ordering — sequence gaps, duplicates & resync

Every event carries a monotonic `seq` integer. Client maintains `lastAppliedSeq`. All decisions below are deterministic and require no server round-trip except the explicit resync path.

Full decision tree

```
onReceive(event):
  if event.seq <= lastAppliedSeq:
    DROP (duplicate or reorder — state already correct)
    return

  if event.seq == lastAppliedSeq + 1:
    APPLY immediately
    lastAppliedSeq = event.seq
    flush pendingBuffer (in-order events waiting)
    return

  if event.seq > lastAppliedSeq + 1:
    BUFFER event (gap detected — missing packets)
    gapTimer = setTimeout(requestResync, 800ms)
    return

requestResync():
  send { type: "STATE_SYNC_REQUEST", fromSeq: lastAppliedSeq }
  host responds with full state snapshot + current seq
  client resets lastAppliedSeq = snapshot.seq
  client clears pendingBuffer
  client re-applies any buffered events with seq > snapshot.seq
```

Concrete example

seq 143	Arrives first. 143 ≠ lastApplied(141)+1 → buffer it, start 800ms gap timer

`seq 142`	Arrives 40ms later. 142 = lastApplied(141)+1 → APPLY, cancel gap timer, flush buffer → APPLY 143
`result`	State applied in correct order. No resync needed. Board never corrupts.

Worst-case path — packet truly lost

```
seq 142 never arrives within 800ms:
  → CLIENT sends STATE_SYNC_REQUEST { fromSeq: 141 }
  → HOST responds STATE_SYNC { state: {...}, seq: 144 }
  → Client rebuilds from snapshot
  → pendingBuffer events > 144 applied on top
  → Board is now consistent

Client shows "Reconnecting..." spinner during the 800ms window.
Spinner dismissed on first successful apply after resync.
Players never see a frozen or corrupted board.
```

Animation queue constraint

Token animations are driven by applied events, not raw incoming events.
Only events that pass the seq gate enter the animation queue.
animationQueue.push(event) is called ONLY from the APPLY branch — never from BUFFER.
This prevents visual glitches from out-of-order renders reaching the player.

The 800ms gap timer covers normal network jitter. On fast LANs, reorder resolves in <50ms. The timer only fires on genuine packet loss.

W3 Mobile touch UX — hit targets, thumb zones, layout

A Ludo board has 52 outer path cells and 4×4 home yard cells. On a 360px-wide phone, each cell is roughly 24px. Tokens sitting inside those cells are even smaller. This is untappable. Every requirement below is a hard constraint, not a preference.

Token tap targets

```
Visible token size: scales with board (may be 16-20px on mobile)
Touch target size: ALWAYS 44 × 44px minimum (Apple HIG + WCAG 2.5.5)

Implementation:
  Each token is an SVG <g> wrapping:
    <circle r="10" />              — visible token
    <circle r="22" fill="transparent" class="touch-target" />  — hit area
```

```
  touch-target circles must never overlap neighboring cell targets.
  If board is smaller than 44px/cell: use Tray Pattern (see below).
```

Dice button — thumb-safe zone

```
Problem: dice button adjacent to board → accidental roll on token tap.

Fix — separate layout zones:
  Mobile portrait layout:
    [ Board (top 60% of screen)         ]
    [ ———————— divider ———————— ]
    [ Dice area (bottom 40%)  ← 64px tall ]

  The dice button itself: min 64 × 64px, centered in dice area.
  Minimum 32px clear margin between board edge and dice area.
  Board touch events and dice touch events in separate DOM containers.
  → Impossible to accidentally trigger one from the other.

  On landscape mobile: dice area moves to sidebar (right 30% of screen).
  Board fills left 70%. Same 32px separation rule applies.
```

Tray pattern — for very small screens (<360px wide)

```
When board cell size < 20px after fitting viewport:
  → Board renders without token visuals on cells
  → A bottom tray shows: [Token 1] [Token 2] [Token 3] [Token 4]
  → Each tray token is 52×52px with clear label (position in steps)
  → Player taps tray token to select it; board highlights valid destination
  → Confirmation tap on board cell completes the move

  Removes the finger-on-tiny-cell problem entirely.
  Tray dismissed when no valid move is available (non-turn players).
```

Required minimums	Accidental input prevention	Landscape mobile	Testing gate
Token touch target: 44×44px Dice button: 64×64px Board–dice gap: 32px minimum Tray tokens: 52×52px	Separate DOM containers for board vs dice 200ms debounce on token tap Confirmation step for moves on <400px screens	Dice → right sidebar (30% width) Board → left zone (70% width) 32px gap enforced	Must pass tap tests on 320px, 360px, 390px viewports before merge. No exceptions.

Deliverable checklist

✓ Modular engine with typed adapter	✓ 60 FPS requestAnimationFrame animations

✓ SVG board, fully responsive (320px–1440px)	✓ Spectator mode + auto host re-election
✓ Host-authoritative sync — RulesEngine on host only	✓ Color-blind accessibility mode
✓ Deterministic host election (W1)	✓ Server-side cheat validation
✓ Hot standby secondary host (W1)	✓ Delta-only sync events (<2KB)
✓ Seq-gated event queue — no state corruption (W2)	✓ 44px token touch targets — mobile (W3)
✓ 800ms gap timer + STATE_SYNC resync (W2)	✓ Tray pattern for <360px screens (W3)